

1

Что такое состояние

Состояние — это данные, которые могут меняться во время работы компонента.



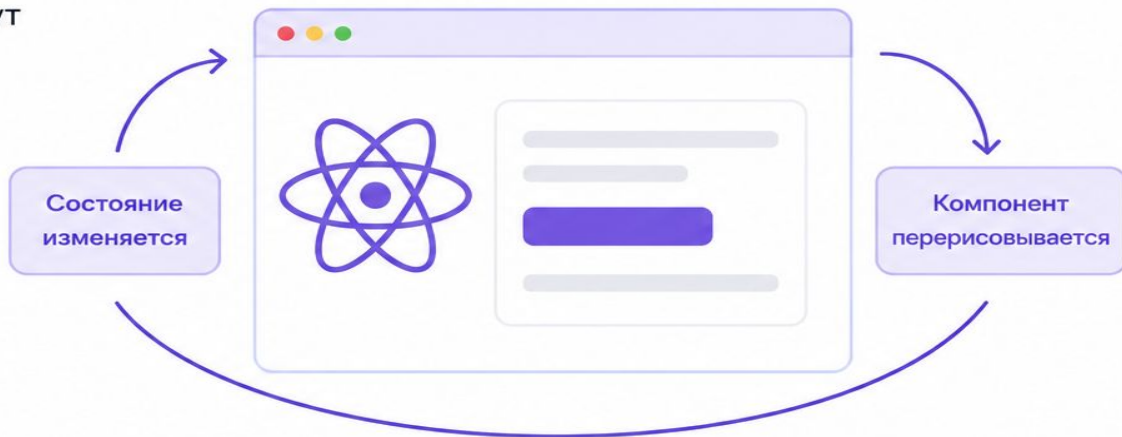
Когда состояние меняется, React заново вызывает компонент.



После этого React обновляет интерфейс.



Это позволяет интерфейсу реагировать на изменения.



Пример

JS

```
const [isProductListVisible, setIsProductListVisible] = useState(false);
```



Ключевая идея

Состояние позволяет компоненту «помнить» информацию и обновлять интерфейс, когда эта информация меняется.

2 Что возвращает useState

useState возвращает массив из двух элементов.

- 1 **Первый элемент** — текущее значение состояния.
- 2 **Второй элемент** — функция для изменения значения.

Синтаксис

```
const [value, setValue] = useState(initialValue);
```

- 1 **value** — текущее значение
- 2 **setValue** — функция для изменения
- 3 **initialValue** — начальное значение



В нашем примере

```
const [isProductListVisible, setIsProductListVisible] =  
  useState(false);
```

- 1 **isProductListVisible** — Текущее значение состояния (тип: **boolean**).
- 2 **setIsProductListVisible** — Функция, которую мы вызываем, чтобы изменить значение.
- 3 **false** — Начальное значение состояния — список товаров скрыт.



Важно

Когда мы вызываем функцию из второго элемента (**setIsProductListVisible**), React обновляет состояние и заново вызывает компонент.

setState(...)

изменяем состояние



React



новый рендер
компонента

обновляем интерфейс

3

Как менять состояние

Чтобы изменить состояние, мы используем **setter** — функцию, которую возвращает **useState**.



Простой вариант

Можно установить конкретное значение:

```
setIsProductListVisible(true);
```



1. Вызываем setter

Мы вызываем `setIsProductListVisible(true)`.



2. React обновляет состояние

React запоминает новое значение (`true`).



3. Компонент перерисовывается

React заново вызывает компонент и обновляет интерфейс.



Пример в коде

```
1  const [isProductListVisible,
2    setIsProductListVisible] = useState(false);
3
4  // Устанавливаем значение в true
5  setIsProductListVisible(true);
6
7  // Устанавливаем значение в false
8  setIsProductListVisible(false);
9
```



Важно помнить

- Значение можно установить любым: `true`, `false`, число, строка, объект и т.д.
- После вызова setter React перерисует компонент.
- Мы не меняем состояние напрямую — только через **setter**.



Ключевая идея

Вызов setter сообщает React, что данные изменились. После этого React сам обновляет компонент и интерфейс.

`setState(...)`

изменяем состояние



React



обновляет интерфейс

4

Обновление от предыдущего значения

Иногда новое значение состояния зависит от старого.

В **setter** передают **функцию (callback)**, которая получает предыдущее значение.

Пример кода

```
1 const handleToggleProductList = () => {  
2   setIsProductListVisible((currentValue) => !currentValue);  
3 };  
4
```

Как это работает

- ✓ `currentValue` — предыдущее значение состояния.
- ✓ `!currentValue` — меняет `true` на `false`, а `false` на `true`.
- ✓ Такой подход безопасен, даже если обновления происходят часто.
- ✓ Удобно для переключателей, счётчиков и других случаев, когда новое значение зависит от старого.



Зачем это нужно?

- React может объединять несколько обновлений состояния.
- Использование callback гарантирует, что мы работаем с актуальным значением.



Как происходит обновление

1 Текущее значение

false

currentValue = false



2 Вычисляем новое

!currentValue

!false = true



3 Новое значение

true

Состояние обновлено

1 Текущее значение

true

currentValue = true



2 Вычисляем новое

!currentValue

!true = false



3 Новое значение

false

Состояние обновлено

Реальный пример использования



Показать / Скрыть список



Состояние

false → true → false → ...

5 Состояние и отрисовка

Интерфейс зависит от текущего значения состояния.

</> Условный рендеринг

С помощью состояния мы решаем, показывать элемент или нет.

```
{isProductListVisible && (  
  <ProductList products={products} />  
)}
```

i Что происходит

- ✓ Если `isProductListVisible === true` — React отрисовывает список товаров.
- ✓ Если `isProductListVisible === false` — список не отрисовывается.
- ✓ UI всегда соответствует текущему значению состояния.

💡 Вывод

Состояние управляет тем, что видит пользователь.
Изменили состояние — изменился интерфейс.

🖥️ Примеры на экране

✓ Состояние: true

→ Список
отображается

isProductListVisible = true

Список товаров



Ноутбук

59 990 ₽



Наушники

4 990 ₽



Мышь

1 490 ₽

✗ Состояние: false

→ Список
не отображается

isProductListVisible = false

Список товаров



Список скрыт

Состояние
(true / false)



Интерфейс
(UI)

6

Частая ошибка

Нельзя изменять состояние напрямую.

Нужно использовать **setter**, который возвращает **useState**.



Неправильно

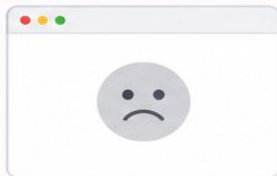
Прямое присваивание не работает с состоянием.

```
1 const [isProductListVisible, setIsProductListVisible] =  
  useState(false);  
2 // Неправильно: прямое изменение состояния  
3 isProductListVisible = true; // ❌ React не узнает об этом
```



Что происходит

- ❌ React не отслеживает такие изменения.
- ❌ Компонент не перерисовывается.
- ❌ Интерфейс не обновляется.



Интерфейс не меняется



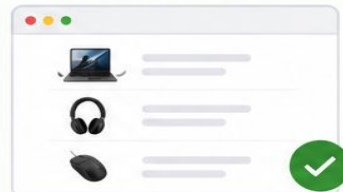
Правильно

Используем setter для изменения состояния.

```
1 const [isProductListVisible, setIsProductListVisible] =  
  useState(false);  
2 // Правильно: используем setter  
3 setIsProductListVisible(true); // ✅ React узнает об этом
```

Что происходит

- ✅ React получает новое значение.
- ✅ Компонент перерисовывается.
- ✅ Интерфейс обновляется.



Интерфейс обновлён



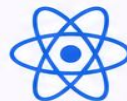
Важно помнить

Состояние в React — это не обычная переменная.
Не изменяйте его напрямую.

Всегда используйте **setter** из **useState**.

useState(value)

вызов setter



React
фиксирует изменение



компонент перерисовывается
интерфейс обновляется

7

Правило 1: хуки только на верхнем уровне

Хуки нельзя вызывать внутри условий, циклов или после `return`.

Они должны вызываться в **одном и том же порядке** при каждом рендере.



Почему это важно



React запоминает хуки по порядку вызова.



Если порядок вызова меняется, React не сможет правильно связать состояние с компонентом.



Поэтому порядок вызова хуков должен быть одинаковым каждый рендер.



Неправильно

Внутри if

```
if (isProductListVisible) {
  const [count, setCount] = useState(0);
}
```

✗ Хук вызывается не всегда.

Внутри цикла

```
items.map((item) => {
  const [open, setOpen] = useState(false);
  return <Item key={item.id} />;
});
```

✗ Хук вызывается разное количество раз.

После return

```
return <div>Привет</div>;
const [value, setValue] = useState('');
```

✗ Ход после return не выполнится.



Правильно

На верхнем уровне компонента

```
const [count, setCount] = useState(0);
if (isProductListVisible) {
  return <Counter count={count} />;
}
```

✓ Хук вызывается всегда.

На верхнем уровне (не в цикле)

```
const [openItems, setOpenItems] =
  useState<Record<number, boolean>>({});
return items.map((item) => (
  <Item key={item.id} open={!!openItems[item.id]} />
));
```

✓ Хук вызывается один раз.

До return

```
const [value, setValue] = useState('');
return <div>{value}</div>;
```

✓ Хук вызывается перед возвратом разметки.



Запомните

- Все хуки вызываются на верхнем уровне компонента.
- Не помещайте хуки внутрь `if`, `циклов`, `try/catch` и после `return`.

`useState()`

1-й хук

+

`useEffect()`

2-й хук

+

`useMemo()`

3-й хук

→



React

→



Интерфейс

Порядок вызова хуков должен быть одинаковым при каждом рендере.

8

Правило 2: хуки только в React-функциях

Хуки можно вызывать только внутри React-функций:

- ✓ в функциональных компонентах
- ✓ в собственных хуках (функции, имя которой начинается с use)

✗ Неправильно

1. В обычной функции

```
1 function getValue() {
2   const [value, setValue] = useState("");
3   return value;
4 }
```

- ✗ `getValue` — обычная функция, а не компонент и не хук.

2. В обработчике события

```
1 function handleClick() {
2   const [count, setCount] = useState(0);
3   // ...
4 }
```

- ✗ Хуки нельзя вызывать внутри обработчиков, условий, циклов и других вложенных функций.

3. В классическом компоненте

```
1 class App extends React.Component {
2   render() {
3     const [value, setValue] = useState("");
4     return <div>{value}</div>;
5   }
6 }
```

- ✗ Хуки работают только в функциональных компонентах.

✓ Правильно

1. В функциональном компоненте

```
1 const App = () => {
2   const [value, setValue] = useState("");
3   return <div>{value}</div>;
4 };
```

- ✓ Хук вызывается внутри функционального компонента. Это правильно.

2. В собственном хуке

```
1 function useInput(initialValue = "") {
2   const [value, setValue] = useState(initialValue);
3   return [value, setValue];
4 }
```

- ✓ Функция начинается с `use` — это пользовательский хук. Внутри него можно вызывать хуки.

3. Использование собственного хука в компоненте

```
1 const App = () => {
2   const [value, setValue] = useInput("");
3   return <input value={value} onChange={e => setValue(e.target.value)} />;
4 };
```

- ✓ Компонент вызывает собственный хук. Это тоже правильно.

i Почему так важно?

React полагается на порядок вызова хуков. Если вызывать хуки в обычных функциях, условиях или обработчиках — порядок может меняться, и состояние «съедет».

★ Что такое пользовательский хук?

Это обычная функция, имя которой начинается с `use`. Она может вызывать другие хуки.

```
1 function useSomething() {
2   const [data, setData] = useState(null);
3   // логика
4   return { data, setData };
5 }
```

💡 Коротко

- ✓ Хуки только в React-функциях.
- ✓ React-функции = компоненты и пользовательские хуки.
- ✓ Обычные функции — нет.



Главная мысль

Хуки предназначены для React. Вызывайте их только внутри компонентов или собственных хуков (`useSomething`).



React-функция
(компонент / хук)



Можно вызывать хуки
`useState()`, `useEffect()` и др.



Нельзя вызывать хуки
в обычных функциях,
условиях, циклах,
обработчиках событий и т.д.

9 Почему правила хуков важны

React запоминает хуки по **порядку их вызова**.

Если порядок вызова меняется между рендерами,

React не сможет правильно связать состояние с компонентом.



Как React хранит хуки

React хранит состояние хуков в списке и сопоставляет каждый хук с ячейкой по индексу. При следующем рендере порядок должен быть точно таким же.

Первый рендер

```
const [a, setA] = useState(0);
const [b, setB] = useState(false);
const [c, setC] = useState("");
```

Внутренний список хуков React

0	0	→ для a
1	false	→ для b
2	""	→ для c

Второй рендер (порядок не изменился)

```
const [a, setA] = useState(0);
const [b, setB] = useState(false);
const [c, setC] = useState("");
```

Внутренний список хуков React

0	0	→ для a
1	false	→ для b
2	""	→ для c



Порядок тот же — состояния связаны правильно.



Что произойдет, если порядок изменится

Первый рендер

0	0	→ для a
1	false	→ для b
2	""	→ для c

Второй рендер (порядок изменился)

```
const [b, setB] = useState(false);
const [a, setA] = useState(0);
const [c, setC] = useState("");
```

Как React сопоставит:

0	0	→ попадёт в b (ошибка!)
1	false	→ попадёт в a (ошибка!)
2	""	→ для c (случайно корректно)



Состояния «переедут» и будут работать неверно.



Вывод

Чтобы React мог правильно сопоставлять хуки с состояниями:

- ✓ Хуки должны вызываться в одном и том же порядке при каждом рендере.
- ✓ Нельзя вызывать хуки внутри условий, циклов и после `return`.
- ✓ Хуки должны вызываться только внутри **React-функций**.



Главная мысль

Стабильный порядок вызова хуков — это то, что позволяет React надежно связывать состояние с компонентом при каждом рендере.

1 = Стабильный
3 = порядок хуков



React



Корректное состояние и предсказуемая работа приложения

10 Практика

Закрепим работу с состоянием и правилами хуков.
Выполните задания и проверьте, что **линтер** не выдает ошибок.



Цель

Научиться управлять состоянием с помощью **useState** и соблюдать правила хуков.

1

Показать и скрыть список



Сделайте кнопку, которая показывает и скрывает список товаров.

Подсказка

Используйте состояние типа **boolean**.

2

Текст кнопки зависит от состояния

Aa

Сделайте так, чтобы текст кнопки менялся в зависимости от того, виден список или нет.

Подсказка

Используйте тернарный оператор или **&&**.

3

Добавить счетчик кликов



Добавьте счетчик кликов по кнопке. Отображайте количество кликов.

Подсказка

Используйте **useState** с начальным значением **0**.

4

Соблюдать правила хуков



Убедитесь, что все хуки вызваны на верхнем уровне и только в React-функциях.

Подсказка

Не вызывайте хуки внутри **if**, циклов и после **return**.

5

Проверить линтер



Убедитесь, что код проходит проверку линтером без ошибок.

Подсказка

Запустите команду **npm run lint**.



Что должно получиться (пример)

Показать список

Кликов: 3

- Ноутбук – 59 990 ₽
- Наушники – 4 990 ₽
- Мышь – 1 490 ₽



Скрыть список

Кликов: 3



Список скрыт



Полезные напоминания

Объявление состояния

```
1 const [isVisible, setIsVisible] = useState(false);  
2 const [count, setCount] = useState(0);
```

Обновление от предыдущего значения

```
1 setIsVisible(prev => !prev);
```

Условный рендеринг

```
1 {isVisible && (  
2   <ProductList products={products} />  
3 )}
```

Увеличение счетчика

```
1 setCount(prev => prev + 1);
```



Важно!

Используйте хуки правильно: только на верхнем уровне и только в компонентах или собственных хуках.



Удачи в практике!

11 Дополнительные правила для хуков

Хуки — это функции, поэтому к ним применяются **особые правила**.

✓ Правильно

Вызывайте хуки на верхнем уровне.

```
1 function Counter() {
2   const [count, setCount] = useState(0);
3   const [name, setName] = useState('');
4   // ...
5   return <div>...</div>;
6 }
```

✓ Хуки вызываются всегда в одном и том же порядке при каждом рендере.

✗ Неправильно

Не вызывайте хуки внутри условий, циклов или вложенных функций.

Внутри условия

```
1 if (isLoggedIn) {
2   const [user] = useState(null);
3 }
```

✗ Хук может вызываться не при каждом рендере.

Внутри цикла

```
1 items.map(item => {
2   const [value] = useState(0);
3   return ...
4 })
```

✗ Хук может вызываться разное количество раз.

Внутри вложенной функции

```
1 function handleClick() {
2   const [count] = useState(0);
3 }
```

✗ Хук не должен вызываться внутри функций.

Почему эти правила необходимы?



React хранит хуки в массиве по порядку вызова.

Если порядок меняется — состояния «сезжают».

1-й рендер

useState(0)
(index 0)

useState('')
(index 1)

useState(false)
(index 2)

Следующий рендер

useState(0)
(index 0)

useState('')
(index 1)

useState(false)
(index 2)



Порядок тот же — состояния связаны правильно.



Пользовательские хуки

Пользовательские хуки также должны следовать правилам. Внутри них можно использовать другие хуки.

```
1 function useUser() {
2   const [user, setUser] = useState(null);
3   useEffect(() => {
4     // ...
5   }, []);
6   return { user, setUser };
7 }
```



Исключение: ранний return

Ранний return допустим, если он до любых хуков.

Правильно

```
1 function Comp({ error }) {
2   if (error) return <Error />;
3   const [count] = useState(0);
4   return <div>{count}</div>;
5 }
```

✓ Хуки всё равно вызываются в одном порядке.

Неправильно

```
1 function Comp({ error }) {
2   const [count] = useState(0);
3   if (error) return <Error />;
4   return <div>{count}</div>;
5 }
```

✗ Хук может быть пропущен при некоторых рендерах.



Коротко

- ✓ Всегда вызывайте хуки на верхнем уровне React-функции.
- ✓ Не вызывайте хуки внутри условий, циклов, вложенных функций и после **return**.
- ✓ Соблюдайте эти правила, чтобы React мог корректно сохранять состояние.

Часто используемые хуки

Хук	Для чего нужен
useState(initialValue)	Хранение состояния
useEffect(effect, deps)	Побочные эффекты
useContext(Context)	Доступ к контексту
useRef(initialValue)	Ссылка на значение / DOM
useMemo(factory, deps)	Кеширование вычислений
useCallback(fn, deps)	Мемоизация функций



Полезные ресурсы

- Официальная документация React react.dev/learn/rendering-lists#rules-of-hooks
 - ESLint плагин `npm i eslint-plugin-react-hooks`
 - Create React App / Vite
- Проверки правил хуков включены по умолчанию.



Главное: Хуки делают компоненты мощными и удобными. Соблюдайте правила — и ваш код будет предсказуемым и стабильным!